NOSQL DATABASE DESIGN – LAB SHEET 4

Solutions: HBase Shell Commands and Cassandra CQL Queries with Sample Outputs

Note: Run HBase commands in the HBase shell and Cassandra statements in `cqlsh`. Outputs below are illustrative; exact formatting and generated timestamps can vary by version. For Cassandra examples, use a local single-node setup unless your lab specifies a cluster.

1. Install/configure HBase; Student table with two column families

After HBase is installed and the HBase service is running, open the HBase shell and execute:

Code / Required Queries

```
create 'Student', 'personal', 'academic'
put 'Student', 'S101', 'personal:name', 'Aarav'
put 'Student', 'S101', 'personal:city', 'Roorkee'
put 'Student', 'S101', 'academic:course', 'BCA'
put 'Student', 'S101', 'academic:cgpa', '9.2'
put 'Student', 'S102', 'personal:name', 'Diya'
put 'Student', 'S102', 'personal:city', 'Haridwar'
put 'Student', 'S102', 'academic:course', 'BCA'
put 'Student', 'S102', 'academic:cgpa', '8.9'
put 'Student', 'S103', 'personal:name', 'Kabir'
put 'Student', 'S103', 'personal:city', 'Dehradun'
put 'Student', 'S103', 'academic:course', 'BCA'
put 'Student', 'S103', 'academic:cgpa', '8.7'
put 'Student', 'S104', 'personal:name', 'Meera'
put 'Student', 'S104', 'personal:city', 'Roorkee'
put 'Student', 'S104', 'academic:course', 'BCA'
put 'Student', 'S104', 'academic:cgpa', '9.1'
put 'Student', 'S105', 'personal:name', 'Rohan'
put 'Student', 'S105', 'personal:city', 'Delhi'
put 'Student', 'S105', 'academic:course', 'BCA'
put 'Student', 'S105', 'academic:cgpa', '8.5'
scan 'Student'
```

Sample Output

```
ROW   COLUMN+CELL
S101  academic:cgpa = 9.2
S101  academic:course = BCA
S101  personal:city = Roorkee
S101  personal:name = Aarav
S102  academic:cgpa = 8.9
... (remaining cells for S102–S105)
5 row(s)
```

HBase stores values by row key and column-family:qualifier. The scan output contains all inserted cells; the abbreviated output above is for readability.

2. HBase Employee CRUD operations

Create a table, insert records, retrieve a row, scan the table, and delete a cell.

Code / Required Queries

```
create 'Employee', 'info', 'job'
put 'Employee', 'E101', 'info:name', 'Anita'
put 'Employee', 'E101', 'info:city', 'Roorkee'
put 'Employee', 'E101', 'job:role', 'Analyst'
put 'Employee', 'E102', 'info:name', 'Rahul'
put 'Employee', 'E102', 'job:role', 'Developer'
get 'Employee', 'E101'
scan 'Employee'
delete 'Employee', 'E101', 'info:city'
get 'Employee', 'E101'
disable 'Employee'
drop 'Employee'
```

Sample Output

```
get 'Employee', 'E101'  -> row E101 contains:
info:name = Anita
info:city = Roorkee
job:role = Analyst

scan 'Employee' -> 2 row(s)

After delete and get:
info:name = Anita
```

```
job:role = Analyst

drop 'Employee' -> table removed
```

The final disable/drop commands remove the example table. Run them only if you no longer need the table.

3. HBase Event Logging System

Use one family for user/event details and another for status/time metadata.

Code / Required Queries

```
create 'EventLog', 'event', 'meta'
put 'EventLog', 'EV001', 'event:user_id', 'U01'
put 'EventLog', 'EV001', 'event:type', 'LOGIN'
put 'EventLog', 'EV001', 'meta:timestamp', '2026-10-16T09:00:00'
put 'EventLog', 'EV001', 'meta:status', 'SUCCESS'
put 'EventLog', 'EV002', 'event:user_id', 'U02'
put 'EventLog', 'EV002', 'event:type', 'PAYMENT'
put 'EventLog', 'EV002', 'meta:timestamp', '2026-10-16T09:05:00'
put 'EventLog', 'EV002', 'meta:status', 'FAILED'
get 'EventLog', 'EV001'
get 'EventLog', 'EV002'
scan 'EventLog'
```

Sample Output

```
ROW EV001
event:type = LOGIN
event:user_id = U01
meta:status = SUCCESS
meta:timestamp = 2026-10-16T09:00:00

ROW EV002
event:type = PAYMENT
event:user_id = U02
meta:status = FAILED
meta:timestamp = 2026-10-16T09:05:00
```

4. HBase Content Management System (CMS)

Insert, update, retrieve, and delete a content record.

Code / Required Queries

```
create 'CMS', 'content', 'author', 'metadata'
put 'CMS', 'C001', 'content:title', 'HBase Basics'
put 'CMS', 'C001', 'content:body', 'Introduction to HBase'
put 'CMS', 'C001', 'author:name', 'Eva'
put 'CMS', 'C001', 'metadata:status', 'Draft'
get 'CMS', 'C001'
# Update an existing cell by writing a new value
put 'CMS', 'C001', 'metadata:status', 'Published'
get 'CMS', 'C001'
# Delete only the body cell
delete 'CMS', 'C001', 'content:body'
get 'CMS', 'C001'
```

Sample Output

```
First get:
content:title = HBase Basics
content:body = Introduction to HBase
author:name = Eva
metadata:status = Draft

After update:
metadata:status = Published

After delete:
content:title = HBase Basics
author:name = Eva
metadata:status = Published
```

In HBase, put updates the current value of a cell (creating a new version internally unless version settings differ).

5. Blogging Platform using HBase

Create three blog rows and retrieve a selected blog by row key.

Code / Required Queries

```
create 'Blog', 'details', 'author', 'comments'
put 'Blog', 'B001', 'details:title', 'Learning NoSQL'
```

```
put 'Blog', 'B001', 'details:category', 'Technology'
put 'Blog', 'B001', 'author:name', 'Aarav'
put 'Blog', 'B001', 'comments:c1', 'Very useful'
put 'Blog', 'B002', 'details:title', 'Study Tips'
put 'Blog', 'B002', 'details:category', 'Education'
put 'Blog', 'B002', 'author:name', 'Diya'
put 'Blog', 'B003', 'details:title', 'Travel Diary'
put 'Blog', 'B003', 'details:category', 'Travel'
put 'Blog', 'B003', 'author:name', 'Kabir'
get 'Blog', 'B002'
scan 'Blog'
```

Sample Output

```
get 'Blog', 'B002':
details:category = Education
details:title = Study Tips
author:name = Diya

scan 'Blog' -> 3 row(s)
```

6. Column-family model and qualifiers in HBase

A family is defined when the table is created. Qualifiers can be added dynamically.

Code / Required Queries

```
create 'Product', 'specs', 'inventory'
put 'Product', 'P01', 'specs:brand', 'Nova'
put 'Product', 'P01', 'specs:color', 'Black'
put 'Product', 'P01', 'specs:weight', '1.2kg'
put 'Product', 'P01', 'inventory:warehouse', 'WH-A'
put 'Product', 'P01', 'inventory:quantity', '25'
get 'Product', 'P01'
describe 'Product'
```

Sample Output

```
get 'Product', 'P01':
specs:brand = Nova
specs:color = Black
specs:weight = 1.2kg
inventory:warehouse = WH-A
inventory:quantity = 25

describe 'Product' shows column families:
specs, inventory
```

Here, specs and inventory are column families; brand, color, weight, warehouse, and quantity are qualifiers.

7. WebsiteCounter page-view counters

Use HBase's atomic increment command. The counter cell is stored as an integer counter.

Code / Required Queries

```
create 'WebsiteCounter', 'views'
# Start or increment each page counter
incr 'WebsiteCounter', 'home', 'views:count', 1
incr 'WebsiteCounter', 'home', 'views:count', 1
incr 'WebsiteCounter', 'about', 'views:count', 1
incr 'WebsiteCounter', 'contact', 'views:count', 1
get 'WebsiteCounter', 'home'
get 'WebsiteCounter', 'about'
get 'WebsiteCounter', 'contact'
count 'WebsiteCounter'
```

Sample Output

```
get home    -> views:count = 2
get about   -> views:count = 1
get contact -> views:count = 1
count 'WebsiteCounter' -> 3 row(s)
```

Each incr adds to the existing counter atomically. If the lab environment reports a counter-type error, create/use the counter column according to the installed HBase version's counter support.

8. HBase user sessions with TTL

TTL is set in seconds. This example uses 60 seconds; expired cells are removed during normal storage cleanup and may not disappear instantly.

Code / Required Queries

```
create 'UserSession', {NAME => 'session', TTL => 60}
put 'UserSession', 'S001', 'session:user_id', 'U101'
put 'UserSession', 'S001', 'session:device', 'Android'
put 'UserSession', 'S002', 'session:user_id', 'U102'
put 'UserSession', 'S002', 'session:device', 'Laptop'
scan 'UserSession'
# Check again after waiting longer than 60 seconds
scan 'UserSession'
describe 'UserSession'
```

Sample Output

```
Immediately after insertion:
S001 -> session:user_id = U101; session:device = Android
S002 -> session:user_id = U102; session:device = Laptop

After TTL has elapsed and cleanup has occurred:
the expired session cells no longer appear in scan output.
```

TTL applies to cells in the specified family. Expiration timing and physical cleanup depend on HBase compaction; do not expect an exact instant removal.

9. Row-key design for horizontal scaling

A simple salted-prefix strategy spreads sequential IDs across multiple key ranges. The salt is calculated by the application before writing.

Code / Required Queries

```
create 'AppRecords', 'data'
# Example keys use a precomputed bucket prefix: 00, 01, 02, 03
put 'AppRecords', '00#1001', 'data:user', 'U01'
put 'AppRecords', '01#1002', 'data:user', 'U02'
put 'AppRecords', '02#1003', 'data:user', 'U03'
put 'AppRecords', '03#1004', 'data:user', 'U04'
scan 'AppRecords'
# Optional: pre-split at creation time in a fresh table:
# create 'AppRecordsSplit', 'data', SPLITS => ['01#', '02#', '03#']
```

Sample Output

```
00#1001 -> data:user = U01
01#1002 -> data:user = U02
02#1003 -> data:user = U03
03#1004 -> data:user = U04
4 row(s)
```

Salting avoids concentrating sequential keys in one region. Pre-splitting can create initial regions; real distribution depends on split keys, data volume, and cluster configuration.

10. HBase regions and RegionServers

Create a pre-split table so HBase has multiple initial regions. RegionServers host and serve regions.

Code / Required Queries

```
# Run on a new table name; split keys define initial region boundaries
create 'RegionalData', 'info', SPLITS => ['m', 't']
put 'RegionalData', 'a001', 'info:value', 'Region range 1'
put 'RegionalData', 'n001', 'info:value', 'Region range 2'
put 'RegionalData', 'z001', 'info:value', 'Region range 3'
scan 'RegionalData'
# Show region assignment/metadata (command availability may vary by HBase version)
status 'detailed'
list
```

Sample Output

```
scan 'RegionalData':
a001 -> info:value = Region range 1
n001 -> info:value = Region range 2
z001 -> info:value = Region range 3

status 'detailed' shows cluster/RegionServer status where supported.
```

Split keys m and t create key ranges below m, m-to-t, and t-or-higher. HBase assigns regions to RegionServers and can split/move regions as data and load grow.

11. Cassandra CollegeDB and Students table

Run these statements in cqlsh. The replication factor 1 is suitable for a single-node lab only.

Code / Required Queries

```
CREATE KEYSPACE CollegeDB
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE CollegeDB;

CREATE TABLE Students (
    student_id int PRIMARY KEY,
    name text,
    course text,
    city text,
    cgpa decimal
);

INSERT INTO Students (student_id, name, course, city, cgpa)
VALUES (101, 'Aarav', 'BCA', 'Roorkee', 9.2);
INSERT INTO Students (student_id, name, course, city, cgpa)
VALUES (102, 'Diya', 'BCA', 'Haridwar', 8.9);
INSERT INTO Students (student_id, name, course, city, cgpa)
VALUES (103, 'Kabir', 'BCA', 'Dehradun', 8.7);
INSERT INTO Students (student_id, name, course, city, cgpa)
VALUES (104, 'Meera', 'BCA', 'Roorkee', 9.1);
INSERT INTO Students (student_id, name, course, city, cgpa)
VALUES (105, 'Rohan', 'BCA', 'Delhi', 8.5);

SELECT * FROM Students;
```

Sample Output

```
student_id | name  | course | city     | cgpa
------------+-------+--------+----------+------
 101        | Aarav | BCA    | Roorkee  | 9.2
 102        | Diya  | BCA    | Haridwar | 8.9
 103        | Kabir | BCA    | Dehradun | 8.7
 104        | Meera | BCA    | Roorkee  | 9.1
 105        | Rohan | BCA    | Delhi    | 8.5

(5 rows)
```

12. Cassandra Employees CRUD operations

Cassandra tables require a primary key. Use the same keyspace as question 11.

Code / Required Queries

```
USE CollegeDB;

CREATE TABLE Employees (
    employee_id int PRIMARY KEY,
    name text,
    department text,
    salary decimal
);

INSERT INTO Employees (employee_id, name, department, salary)
VALUES (1, 'Anita', 'Analytics', 35000);
INSERT INTO Employees (employee_id, name, department, salary)
VALUES (2, 'Rahul', 'IT', 40000);

SELECT * FROM Employees;
UPDATE Employees SET salary = 38000 WHERE employee_id = 1;
SELECT * FROM Employees WHERE employee_id = 1;
DELETE FROM Employees WHERE employee_id = 2;
SELECT * FROM Employees;
```

Sample Output

```
Initial SELECT -> 2 rows:
1 | Anita | Analytics | 35000
2 | Rahul | IT        | 40000

After UPDATE:
1 | Anita | Analytics | 38000

After DELETE:
1 | Anita | Analytics | 38000
```

13. Cassandra event logging by user and time period

Use a composite primary key with user_id as the partition key and event_time as the clustering column. This supports time-range queries for one user.

Code / Required Queries

```
USE CollegeDB;

CREATE TABLE UserEvents (
    user_id text,
    event_time timestamp,
    event_id text,
    event_type text,
    status text,
    PRIMARY KEY (user_id, event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);

INSERT INTO UserEvents (user_id, event_time, event_id, event_type, status)
VALUES ('U01', '2026-10-16 09:00:00', 'EV01', 'LOGIN', 'SUCCESS');
INSERT INTO UserEvents (user_id, event_time, event_id, event_type, status)
VALUES ('U01', '2026-10-16 09:15:00', 'EV02', 'PAYMENT', 'SUCCESS');
INSERT INTO UserEvents (user_id, event_time, event_id, event_type, status)
VALUES ('U01', '2026-10-16 10:00:00', 'EV03', 'LOGOUT', 'SUCCESS');
INSERT INTO UserEvents (user_id, event_time, event_id, event_type, status)
VALUES ('U02', '2026-10-16 09:10:00', 'EV04', 'LOGIN', 'FAILED');

SELECT * FROM UserEvents
WHERE user_id = 'U01'
AND event_time >= '2026-10-16 09:00:00'
AND event_time <= '2026-10-16 09:30:00';
```

Sample Output

```
user_id | event_time           | event_id | event_type | status
---------+----------------------+----------+------------+--------
 U01     | 2026-10-16 09:15:00  | EV02     | PAYMENT    | SUCCESS
 U01     | 2026-10-16 09:00:00  | EV01     | LOGIN      | SUCCESS

(2 rows)
```

Cassandra generally requires partition-key restrictions for efficient queries. The table is designed specifically for user-and-time retrieval rather than arbitrary searches.

14. Cassandra consistency levels ONE, QUORUM, ALL

Create a replicated keyspace for a multi-node cluster. A replication factor of 3 requires at least three suitable replicas for ALL to succeed.

Code / Required Queries

```
CREATE KEYSPACE ConsistencyDemo
WITH replication = {'class': 'NetworkTopologyStrategy', 'datacenter1': 3};

USE ConsistencyDemo;

CREATE TABLE Messages (
    message_id int PRIMARY KEY,
    message text
);

CONSISTENCY ONE;
INSERT INTO Messages (message_id, message) VALUES (1, 'Hello Cassandra');
SELECT * FROM Messages WHERE message_id = 1;

CONSISTENCY QUORUM;
SELECT * FROM Messages WHERE message_id = 1;

CONSISTENCY ALL;
SELECT * FROM Messages WHERE message_id = 1;
```

Sample Output

```
CONSISTENCY ONE:
Query may succeed when one replica responds.

CONSISTENCY QUORUM:
Query waits for a majority of replicas to respond.

CONSISTENCY ALL:
Query requires all replicas for the requested operation; it can fail if any required replica is unavailable.
```

On a single-node lab, do not use replication factor 3 for actual testing: the required replicas do not exist, so QUORUM/ALL operations may fail. Use a three-node test cluster to observe these levels properly.

15. Column-oriented NoSQL application: Event Logging using Cassandra

This complete mini-application demonstrates table creation, insert, query, update, and delete. It uses Cassandra CQL.

Code / Required Queries

```
CREATE KEYSPACE EventApp
WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

USE EventApp;

CREATE TABLE Events (
    user_id text,
    event_time timestamp,
    event_id text,
    event_type text,
    status text,
    PRIMARY KEY (user_id, event_time, event_id)
) WITH CLUSTERING ORDER BY (event_time DESC);

INSERT INTO Events (user_id, event_time, event_id, event_type, status)
VALUES ('U10', '2026-10-16 11:00:00', 'E100', 'LOGIN', 'SUCCESS');

INSERT INTO Events (user_id, event_time, event_id, event_type, status)
VALUES ('U10', '2026-10-16 11:05:00', 'E101', 'UPLOAD', 'FAILED');

SELECT * FROM Events WHERE user_id = 'U10';

UPDATE Events SET status = 'SUCCESS'
WHERE user_id = 'U10'
AND event_time = '2026-10-16 11:05:00'
AND event_id = 'E101';

SELECT * FROM Events WHERE user_id = 'U10';

DELETE FROM Events
WHERE user_id = 'U10'
AND event_time = '2026-10-16 11:00:00'
AND event_id = 'E100';

SELECT * FROM Events WHERE user_id = 'U10';
```

Sample Output

```
First SELECT:
U10 | 2026-10-16 11:05:00 | E101 | UPLOAD | FAILED
U10 | 2026-10-16 11:00:00 | E100 | LOGIN  | SUCCESS

After UPDATE:
E101 status = SUCCESS

After DELETE:
Only E101 remains for user U10.
```

Consistency: tunable per operation (for example ONE or QUORUM). Availability: depends on replica availability and chosen consistency level. Transactions: Cassandra supports atomicity at the single-partition level, not general multi-row ACID transactions. Query features: CQL works best when queries match the primary-key design. Scaling: partitioning distributes data across nodes; add nodes and use a suitable replication strategy for production.